Use of AI assistants. Each problem carries one of three marks, No AI, AI allowed or AI recommended. Whatever you use an assistant for, the code is yours to explain and to change at the defense. Answers go in a single Racket file, built from the template, and `transcript.md` beside it holds the whole exchange, verbatim and unedited, for every problem where you used an assistant.

Motivation

Consider a view that a colleague defined last month, and a query that you write today on top of it. Here we use PostgreSQL syntax:

```
CREATE VIEW by_length AS
  SELECT * FROM films ORDER BY minutes;

SELECT title FROM by_length
  WHERE rating > 8.1 ORDER BY minutes LIMIT 2;
```

The view suggests sorting the films before filtering them. A database planner can instead apply the rating filter before sorting, as in

```
SELECT title FROM films
  WHERE rating > 8.1 ORDER BY minutes LIMIT 2;
```

Suppose the table contains a million films, but only four pass the filter. Then the sort processes four films instead of a million. Note that the outer query needs its own `ORDER BY`: ordering inside a view alone does not guarantee the order of its results.

A query may also be written as a chain of calls, for example in Django:

```
Film.objects.order_by("minutes").filter(rating__gt=8.1)[:2]
```

Django builds a query without running it, then sends it to the database, whose planner chooses how to execute it. Many LINQ operations also defer execution, although deferred execution alone does not imply query rewriting. Ordinary pandas operations, by contrast, compute their results eagerly.

Execution can produce rows on demand where the stages allow it. A `LIMIT` stops after the required number of results, but a sort must first read its whole input unless an index already supplies the required order. In this project, you will build a library in Racket that combines queries as data, an evaluator, execution over streams, an optimiser made of rewrite rules, and a plan display with measured row counts. The filter moved below the sort is the third of the four rules in section 3.

1 A query is data

15 points

Consider an online cinema app that has a collection of films and a schedule of showings. A film has a title, a year, a length in minutes and a rating. A showing has the title of the film, a channel and a start time, written as `hhmm`, so that `1830` is half past six in the evening. The template gives the two structs, seven films, eight showings, and five small helpers used in the examples below:

```
(define (high? f) (> (film-rating f) 8.1))
(define (recent? f) (> (film-year f) 1975))
(define (evening? s) (>= (showing-start s) 1800))
(define (same-title? f s) (equal? (film-title f) (showing-title s)))
(define (title+channel p)
  (list (film-title (car p)) (showing-channel (cdr p))))
```

In this section, you will implement an evaluator for queries. A query is a value, built from six structs given in the template:

```
(struct table (rows))                ; the rows themselves
(struct where (test source))         ; the rows of source that pass test
(struct select (function source))    ; function of each row of source
(struct order-by (key source))       ; the rows of source, by key, ascending
(struct limit (count source))        ; the first count rows of source, or fewer
(struct join (test left right))      ; every pair (cons l r) with (test l r) true
```

Note that nothing is computed when a query is built: `(where high? (table films))` is a struct with a function and another struct inside. The rows appear only when the query is run.

Throughout the project, assume that tests, projections and keys are pure functions, defined on every row they may receive. Tests return booleans, keys return real numbers other than NaN, and counts are non-negative integers. A table may contain rows of any kind, provided the functions in the query accept them. For `run`, `cost` and `explain`, every table must contain a finite list. Section 2 also allows streams. Diagnostic printing in sections 2 and 5 counts which rows are read or which functions are called; it is separate from the pure functions used by a query.

Problem 1.1. No AI Implement a function `run` such that `(run q)` produces the list of rows of a query `q` built from `table`, `where` and `select`. Use recursion on the query: a **`cond`** with one clause per struct, each running the inner query first.

```
(length (run (table films)))                          ; 7
(run (select film-title (where high? (table films))))
; '("Stalker" "Kin-dza-dza!" "Ivan Vasilievich" "The Diamond Arm")
(length (run (where recent? (table films))))          ; 4
```

Problem 1.2. No AI Extend `run` to `order-by` and `limit`. Note that if the count exceeds the number of rows, all available rows are returned. You may use `sort` with a key, given in the template.

```
(run (limit 2 (select film-title (where high? (table films)))))
; '("Stalker" "Kin-dza-dza!")
(run (limit 3 (select film-title (order-by film-minutes (table films)))))
; '("Courier" "Ivan Vasilievich" "The Diamond Arm")
(length (run (limit 10 (table films))))               ; 7
```

Problem 1.3. No AI Extend `run` to `join`: every pair of a row of the left query and a row of the right query that the test accepts, as `(cons l r)`. Preserve the order of the left rows. For each left row, preserve the order of the matching right rows.

```
(length (run (join same-title? (table films) (table showings)))) ; 8
(run (select title+channel
             (join same-title?
                   (where high? (table films))
                   (where evening? (table showings)))))
; '(("Stalker" 1) ("Kin-dza-dza!" 1))
```

2 On demand

20 points

`run` builds every intermediate list in full. In this section, you will implement a version that produces its rows on demand, as a stream. A `limit` over a table can then stop after the required number of rows. Recall the test of lecture 4: would the function still work on an input with no end? Every list can be used as a stream, so the new evaluator accepts the same queries as `run`. It also accepts tables whose rows are streams, possibly with no end. An `order-by` needs a finite source, and we require the right source of a `join` to be finite.

Problem 2.1. AI allowed Implement a function `run/stream` such that `(run/stream q)` produces the rows of `q` as a stream. Where `run` used `filter` and `map`, use `stream-filter` and `stream-map`. For `limit`, write a helper that stops when the count is zero or the stream is empty, using **`stream-cons`** to produce a row and delay the rest. For `join`, use recursion on the left stream and `stream-append` to put the pairs of one left row before those of the rest. Note that `stream-append` is a function: it evaluates its arguments before appending them. Wrap the recursive call in `stream-lazy`, which delays an expression that produces a stream, just as **`stream-cons`** delays its tail. For `order-by`, turn the inner stream into a list, sort it, and return the list (see 2.2).

```
(stream->list
 (run/stream (limit 2 (select film-title (where high? (table films))))))
; '("Stalker" "Kin-dza-dza!")
(stream->list (run/stream (limit 3 (where even? (table (from 1))))))
; '(2 4 6)
(stream->list (run/stream (limit 10 (table '(1 2))))) ; '(1 2)
(stream->list (run/stream (limit 1 (table '()))))     ; '()
(stream->list
 (run/stream (limit 2 (join < (table (from 1)) (table '(3))))))
; '((1 . 3) (2 . 3))
```

Problem 2.2. No AI Explain in two sentences why `order-by` cannot produce its rows on demand. Use the test of the lecture.

Problem 2.3. No AI The template gives `loud`, which turns the list of films into a stream that prints a line each time a row is read from it. Run the two expressions below.

```
(stream->list
 (run/stream (limit 2 (select film-title (where high? (table (loud films)))))))
(run (limit 2
            (select film-title
                    (where high? (table (stream->list (loud films)))))))
```

(a) How many lines does each expression print?

(b) Explain the two numbers, in one sentence each.

3 The optimiser

30 points

In this section, you will implement an optimiser for queries. Since a query is data, a rewrite is a function from a query to a query, and it is safe when it preserves the rows and their order under the assumptions of section 1. A query is a tree: each non-table struct holds one or two source queries, so the outermost struct is the *root*, and the tables are the leaves. We use four rules, each at the root of a query, that is, on the outermost struct and the struct directly inside it:

```
(where p (where q x))   →  (where (lambda (r) (and (q r) (p r))) x)
(select f (select g x)) →  (select (lambda (r) (f (g r))) x)
(where p (order-by k x)) →  (order-by k (where p x))
(limit n (select f x))  →  (select f (limit n x))
```

The first two merge two stages into one. The third sorts only the rows that survive the test. The fourth applies `f` to the rows that are kept rather than to all of them when using `run`; the stream evaluator already applies projections on demand. The template gives `size`, the number of stages including tables, and two queries with redundant stages:

```
(define slow-recent
  (where recent? (where high? (order-by film-minutes (table films)))))
(define slow-two
  (limit 2 (select film-title (select keep (where high? (table films))))))
```

Problem 3.1. No AI Implement a function `rule` such that `(rule q)` applies whichever of the four rules matches the root of `q`, that is, its outermost struct and the one directly inside it, and produces `q` itself when none does. Note that `rule` looks at the root only; 3.2 takes it to the rest of the tree.

```
(size (rule (where recent? (where high? (table films)))))       ; 2
(size (rule (select film-title (select keep (table films))))) ; 2
(size (rule (where high? (order-by film-minutes (table films)))))
; 3; order-by is now outside the where
(size (rule (limit 2 (select film-title (table films)))))
; 3; select is now outside the limit
(size (rule (limit 2 (where high? (table films)))))           ; 3, unchanged
```

Problem 3.2. AI allowed Implement a function `rewrite` such that `(rewrite q)` applies `rule` at every node of the tree of `q`, leaves first. Use recursion on the query: rebuild each struct around its rewritten inner query, then apply `rule` to the result.

```
(size (rewrite slow-recent))
; 4; both wheres are now below the order-by
(size (rewrite slow-two))          ; 4, the selects fused
```

Problem 3.3. AI allowed Implement `optimise` such that `(optimise q)` applies `rewrite` to `q` as many times as `q` has stages, and produces the result. Then explain in one sentence why that many passes are enough.

```
(size (optimise slow-recent))  ; 3
(run (optimise slow-recent))   ; the same rows as (run slow-recent)
(size (optimise slow-two))     ; 4
(run (optimise slow-two))      ; '("Stalker" "Kin-dza-dza!")
```

Problem 3.4. No AI Implement a function `cost` such that `(cost q)` is the sum of the numbers of rows produced by its non-table stages, computed by running each stage. This is a simple cost model for finite list queries; it does not count comparisons or measure execution time.

```
(cost slow-recent)             ; 13
(cost (optimise slow-recent))  ; 4
(cost slow-two)                ; 14
(cost (optimise slow-two))     ; 8
```

Problem 3.5. AI allowed Database systems such as PostgreSQL provide `EXPLAIN`, which displays a query plan and estimated costs, and `EXPLAIN ANALYZE`, which executes the query and reports statistics such as times and row counts. Here we build a simpler display of the measured row counts. Implement a function `explain` such that `(explain q)` prints one line per stage of `q`, the root first, each indented by two spaces per level and followed by the number of rows it produces. Run each stage to count its rows; you need not instrument a single execution of the whole query. The template gives `stage-name`, `sources`, and the string helpers.

```
(explain slow-recent)
; where: 2 rows
;   where: 4 rows
;     order-by: 7 rows
;       table: 7 rows
(explain (optimise slow-recent))
; order-by: 2 rows
;   where: 2 rows
;     table: 7 rows
```

Problem 3.6. No AI Which of the four rules is the claim you proved in problem set 3? Answer in one sentence, naming the claim.

Problem 3.7. No AI Explain in two sentences why the third rule is safe, that is, why the two sides of the rule produce the same rows in the same order.

4 Three questions

15 points

The library is for asking questions. In this section, you will write each of the three questions below twice: as a query, run with `run`, and as a `for` form over the lists directly. The two must agree.

Problem 4.1. No AI Which channels show a film rated above `8.1` in the evening? Produce each answer as the title and the channel. Name the query `evening-channels` and the function `evening-channels/for`.

```
(run evening-channels)  ; '(("Stalker" 1) ("Kin-dza-dza!" 1))
(evening-channels/for)  ; the same
```

Problem 4.2. AI allowed What are the three shortest films with a showing on channel 1, shortest first? Produce the titles. Name the query `shortest-on-1` and the function `shortest-on-1/for`. Count each matching showing separately, so a film with several showings may appear more than once. On equal lengths, preserve the order of the matching pairs from `join`.

```
(run shortest-on-1)  ; '("The Diamond Arm" "Kin-dza-dza!" "Stalker")
(shortest-on-1/for)  ; the same
```

Problem 4.3. AI allowed What are the two best-rated films with an evening showing, best first, and on which channels? Produce each answer as the title and the channel. Name the query `best-evening` and the function `best-evening/for`. Note that `order-by` sorts ascending, and that the key may be any real number other than NaN.

```
(run best-evening)  ; '(("Kin-dza-dza!" 1) ("Stalker" 1))
(best-evening/for)  ; the same
```

Problem 4.4. No AI For each of the three questions, say in one sentence which of the two ways reads better, and why.

5 Three more rules

10 points

In this section, you will ask an assistant for three more rewrite rules, meant to be correct, and then find out whether they are. A rewrite rule is safe only when it preserves the rows and their order on every query satisfying the assumptions of section 1. A rule that passes the examples on this sheet may still be wrong. An assistant may suggest plausible but incorrect rules, so check each rule yourself, including its claim to reduce the work.

Problem 5.1. AI recommended Ask an assistant for three more rewrite rules, starting from the following prompt, pasted as it is:

> I have a small query library in Racket. A query is one of six structs:
>
> ```
> (struct table (rows))
> (struct where (test source))
> (struct select (function source))
> (struct order-by (key source))
> (struct limit (count source))
> (struct join (test left right))
> ```
>
> A function `run` turns a query into its list of rows. I already have four rewrite rules that keep the rows the same: two nested `where`s merge into one, two nested `select`s merge into one, a `where` above an `order-by` moves below it, and a `limit` above a `select` moves below it. Suggest three more rewrite rules that make a query cheaper to run, each as a Racket function from a query to a query that returns the query unchanged when the rule does not apply. Say for each rule why the rows come out the same.

Then continue the conversation until you have three rules as code.

- If a rule is one of the four above, say so and ask for a different one.
- If the assistant gives prose without code, ask for the function.
- If the assistant claims that a rule is safe, ask on what query it would fail, and record the answer. Check the claim yourself.
- If a rule moves a `limit` past a `where`, ask what happens when the test drops the first row. If it moves a `limit` past an `order-by`, ask what happens when the smallest key belongs to a row beyond the limit.
- If a rule needs something the library does not have, such as a look inside a test, say what the library has and ask again.
- If the assistant returns nothing, send a short nudge, such as a question mark, and record that too.

Put the three rules into the answer file as `rule/a`, `rule/b` and `rule/c`. Use a basic, free model; whatever mode it chooses on its own is fine, and the transcript shows which.

Problem 5.2. No AI For each of the three rules, either give a query on which the rule produces different rows from `run` of the original, shown by running both in the answer file, or explain in two sentences why the rule is safe on every query.

6 The audit

10 points

Problem 6.1. No AI Answer in your own words, one or two sentences each. Answers in this section must be written without an assistant. Any code obtained from an assistant must be run on the examples in the template before submission.

(a) What did you ask the assistant to do, and in which problems?

(b) What came back wrong, and how did you find it?

(c) Which example or check in your file catches it now?

(d) Which problem would you not trust an assistant with, and why?